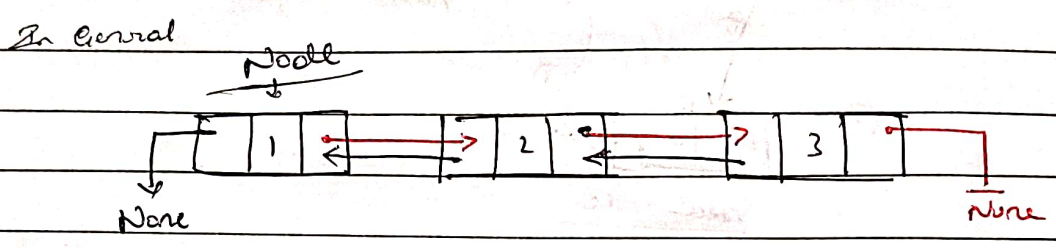
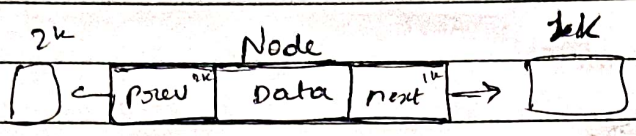
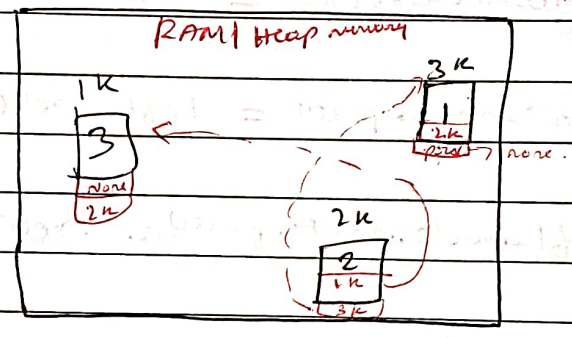


X3 Doubly linked list in python - create & insert

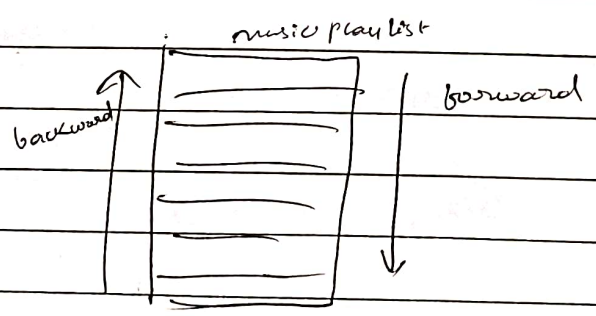


How DLL see in RAM



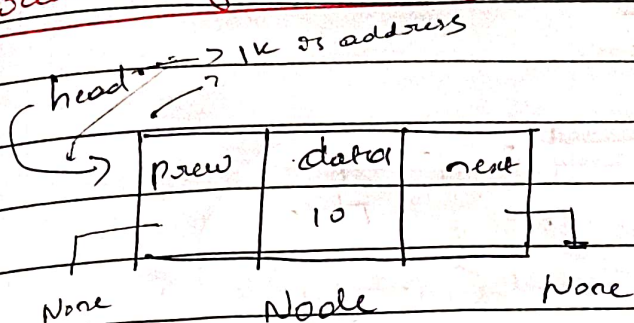
Advantage :-

- ① SLL → uni direct but doubly LL . have 2 direct



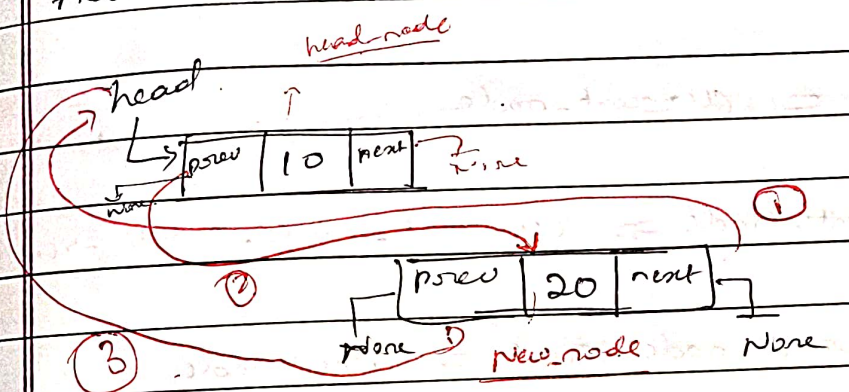
LRU cache implementation (Imp in Interview)

Creation of DLL :-



Insert node at Beginning :-

Added a New Node (20) at beginning.

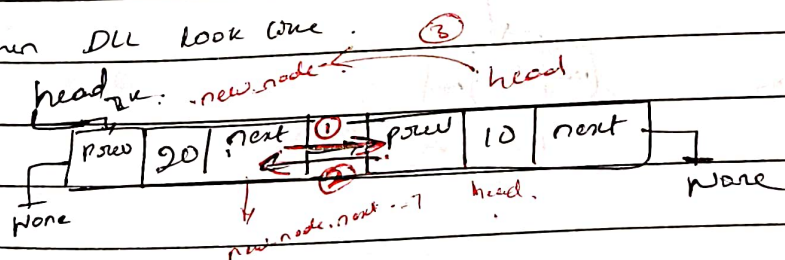


1 $\text{new_node.next} = \text{head}$

2 $\text{head.prev} = \text{new_node}$

3 $\text{head} = \text{new_node}$

Then DLL look like :

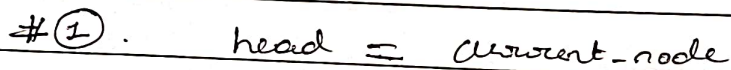


When head is empty / none ($\text{head} \rightarrow \text{none}$)

$\text{head} = \text{new_node}$

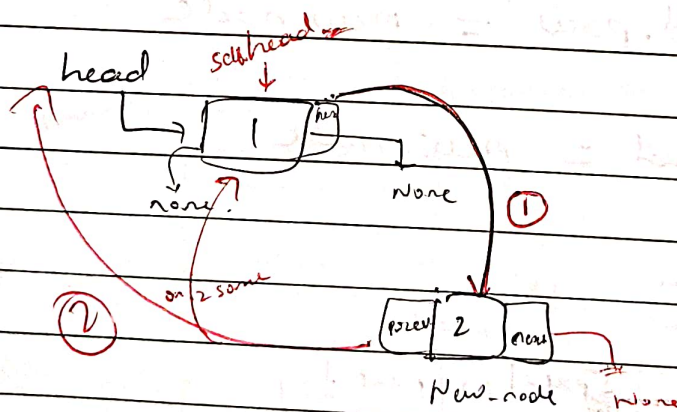
Ans: tail pointer.

⌘ head



④ ③ $\text{current_node} \rightarrow \text{next} = \text{new_node}$

for 1 node 1




```

1  class Node:
2      def __init__(self,data):
3          self.data = data
4          self.prev = None
5          self.next = None
6
7  class DoublyLinkedList:
8      def __init__(self):
9          self.head = None
10
11     def insert_at_beginning(self,data):
12         new_node = Node(data)
13
14         # list is empty
15         if self.head is None:
16             self.head = new_node
17             return
18
19
20         # 1 or more nodes
21         new_node.next = self.head           # new_node.next -> head
22         self.head.prev = new_node          # head.prev -> new_node
23         self.head = new_node               # self.head -> new_node
24
25     def insert_at_end(self,data):
26         new_node = Node(data)
27
28         # list is empty
29         if self.head is None:
30             self.head = new_node
31             return
32
33         # for 1 node only
34         if self.head.next is None:
35             self.head.next = new_node
36             new_node.prev = self.head
37             return

```



```
7 class DoublyLinkedList:
25     def insert_at_end(self,data):
39         # 2 or more nodes
40         current_node = self.head
41
42         while current_node.next is not None:
43             current_node = current_node.next
44
45         current_node.next = new_node
46         new_node.prev = current_node
47
48
49
50     def print_list(self):
51
52         current_node = self.head
53
54         while current_node is not None:
55             print(current_node.data , end = " <-> ")
56             current_node = current_node.next
57
58         print("None")
59
60     # driver code to test the above class
61 > def insert_at_beginning_test():...
68
69     def insert_at_end_test():
70         dll = DoublyLinkedList()
71         dll.insert_at_end(10)
72         dll.insert_at_end(20)
73         dll.insert_at_end(30)
74
75         dll.print_list()
76 if __name__ == "__main__":
77
78     # insert_at_beginning_test()
79     insert_at_end_test()
```